

Library management system with book research ▾

Let's make a Library Management Page using Java that uses OOP, LinkedList, and ArrayList. That will add or remove books. Make it connected to a public library. But also, research any books I want to read online, + the price of the books, or even a PDF version of them. It should also store the title, author, ISBN, Publisher, Published Date, Description, page count, and rating.

Reading frontend design skill >

Now let me build this comprehensive Library Management System as a React artifact with the Anthropic API powering book research:

All Books

LinkedList<Book> ·
ArrayList<Book>

+ Add Book

🔍 Research Online

CATALOG

📖 All Books

+ Add Book

DISCOVERY

🔍 Research Books

DATA STRUCTURES

📄 LinkedList / ArrayList

🔍 Search title, author, ISBN...

All Genres ▾

All Status ▾

Sort: Title ▾

TITLE	AUTHOR	ISBN	GENRE	STATUS	RATING	ACTIONS
Atomi...	Ja...	978-0-7352-1129-2	Self-Help	reserved	★ ★ ★ 4.7	👁️ (
Projec...	And...	978-0-593-13520-4	Sci-Fi	available	★ ★ ★ 4.9	👁️ (
Sapiens	Yuv...	978-0-06-231609-7	Non-Fiction	checked out	★ ★ ★ 4.6	👁️ (
The N...	Pat...	978-0-7564-0407-9	Fantasy	available	★ ★ ★ 4.8	👁️ (

Total Books	4
Available	2
Checked Out	1
Genres	4

Here's your full **LibraryOS** — a Java OOP-style Library Management System! Here's what's packed in:

Data Structures (Java-Style)

- `LinkedList<Book>` — doubly-linked list with head/tail pointers, O(1) add/remove at ends, used for insertion-order tracking
- `ArrayList<Book>` — dynamic array for O(1) index access and catalog browsing

- Both structures stay in sync — every `addBook()` / `removeBook()` updates both

OOP Design

- `Book` class with all 10 fields: title, author, ISBN, publisher, published date, description, genre, page count, rating, and price
- `ListNode` class wrapping each Book for the linked list
- `LinkedList` class with full doubly-linked logic
- `Library` class orchestrating both data structures

Features

- **All Books** — searchable, filterable, sortable catalog table with status badges and star ratings
- **Add Book** — full form for all fields including PDF/online link
- **Research Online** — type any book title or author, and Claude AI searches the web and returns real book data including price, PDF availability, ratings, and buy/preview links — then one-click adds it to your library
- **Data Structures panel** — live visual of what's in your LinkedList and ArrayList as books are added
- **Sidebar stats** — live count of total, available, checked out, and genres

Click **Research Online**, type something like `"The Hobbit"` or `"Andy Weir sci-fi"`, and hit Research ↗ to see it in action!

Let's make a *Library Management Page* in Java that uses OOP, LinkedList, and ArrayList. </br>

That will add or remove books. Make it connected to a public library. But also, research any books that I want </br>

to research online + The price of the books or even a PDF version of it.

It should also store the title, author, ISBN, Publisher, Published Date, Description, page count, and rating.

We should also put it into files.

Check available skills >

Now let me build all the Java files:

12 fichiers créés, a exécuté une commande >

Now let's compile and verify everything works:

Exécuté 7 commandes >

All 11 classes compiled. Now let's do a quick smoke test:

Exécuté 2 commandes >

All working perfectly. Now update the run scripts to include the UTF-8 flag, then package everything:

2 fichiers modifiés, a exécuté une commande >

Everything is clean. Now zip and package for download:

Exécuté 2 commandes, lu un fichier >

Here's the complete **Library Management System in Java** — compiled, tested, and zipped. Here's a full breakdown:

File Structure

```
LibraryManagement/
├── src/library/
│   ├── Main.java           ← Boot sequence
│   ├── model/
│   │   ├── Book.java       ← OOP entity (13 fields, CSV
│   │   │                   serialization)
│   │   ├── BookStatus.java ← Enum: AVAILABLE / CHECKED_OUT /
│   │   │                   RESERVED / LOST
│   │   └── datastructures/
│   │       └── DoublyLinkedList.java ← Custom generic linked list with inner
│   │       │                   Node<T>
│   └── service/
│       ├── Library.java    ← Core logic; owns both data structures
│       │                   in sync
│       ├── FileService.java ← CSV read/write, backups, logging
│       └── BookResearcher.java ← Live HTTP calls to Google Books +
Open Library
└── ui/
```

| | └─ ConsoleUI.java

| └─ util/

| └─ DataSeeder.java

└─ run.sh / run.bat

└─ README.md

← All terminal menus and input

← Seeds 8 sample books on first run

← One-click build & run

OOP Design

`Book` is a fully encapsulated entity with private fields and getters/setters.

`DoublyLinkedList<T>` is a custom generic class with an inner `Node<T>` (prev/next pointers). `Library` owns **both** structures and keeps them in sync — `linkedCatalog` for insertion-order and queue ops, `arrayCatalog` for index access and sorting.

`FileService`, `BookResearcher`, and `ConsoleUI` are each single-responsibility classes wired together in `Main`.

Data Files (auto-created in `data/`)

File	Purpose
<code>books.csv</code>	Main catalog — auto-saved on every change
<code>books_backup_YYYYMMDD_HHmmss.csv</code>	Timestamped backup (last 5 kept)
<code>library.log</code>	Append-only operation log
<code>research_results.txt</code>	Every online search result saved
<code>export_*.csv</code>	User-triggered filtered exports

How to Run

macOS/Linux:

```
bash
chmod +x run.sh && ./run.sh
```

Windows:

```
bat
```

```
run.bat
```

Manual:

```
bash
```

```
find src -name "*.java" | xargs javac -encoding UTF-8 -d out --release 17  
java -Dfile.encoding=UTF-8 -cp out library.Main
```

Requires **JDK 17+**.



Librarymanagement
ZIP

Télécharger

